```
        musicians = Musician.objects.filter(q)

    data = {
        "search_text": search_text,
        "musicians": musicians,
    }

    if request.htmx:
        return render(request, "partials/musician_results.html", data)

    return render(request, "search_musicians.html", data)
```

If this is an HTMX call, render only the partial page.

That's it! Now, you have search-as-you-type. The equivalent in Vue.js requires a couple dozen lines of JavaScript. With HTMX, you add a few attributes, write a search view, and you're there. Go back to the development server, and this time, type your search terms in the input box to test it out. Note how the URL updates as you enter your terms.

Other django-htmx features

In this chapter, I'll only be using the `request.htmx` object as a Boolean to determine if the call is HTMX or not, but the library also has the following:

- Full access to all the headers HTMX sends, including which URL got called and why it got triggered
- An extension JavaScript library that enhances HTMX's error reporting and interaction with the template engine
- Response classes allowing you to perform redirects, refreshes, and other actions as part of HTMX-triggered views

Let's add another feature to your search page: infinite scroll. If your site has a lot of data, you might paginate the results. With infinite scroll, instead of forcing the user to click to get the next page of data, they simply scroll down, and the new data gets added to the bottom. Two things need to be done to add infinite scroll to your existing search feature: first, you need to paginate the data in your view, and second, you need an HTMX trigger that renders the next "page" of data.

You've seen pagination before. It was used in the `musicians()` view way back in chapter 4. As a quick recap, Django provides the `Paginator` object that divides query results into pages of data, and then you use a query parameter to specify which page is displayed. The following listing contains the modifications to the search musicians view to add pagination. It takes advantage of the existing `_get_items_per_page()` and `_get_page_num()` utilities, written in chapter 4's exercises.

Listing 12.10 An HTMX-capable search view

```
# RiffMates/bands/views.py
...
from time import sleep
...
```